QA Robustness to Adversarial Distractors

CortiqoLabs:NLP/GenAI Track

Write-up by Harsh Yadav

Dataset

I used **SQuAD 2.0**, a well-established, publicly available QA dataset created by researchers at Stanford University. It was chosen for this exercise because before SQuAD 2.0, older question answering AI models were forced to guess an answer, even if the information was not in the text. This caused models to make up facts. SQuAD 2.0 makes AI act more like a real person by forcing it to say ”I don’t know” when it cannot find a valid answer. It acts as a trap for models that rely entirely on superficial pattern matching. When an unanswerable question shares identical words with the text, a weak model will blindly extract a nearby phrase, thinking it found the answer.

Hypothesis

Retrieval Augmented QA pipelines, which are usually built on lightweight LLMs (I used Flan T-5), are evaluated on whether they can find the right or the **”golden”** answer. We completely tend to overlook the fact that whether they can defend it or not. I hypothesize that **we can trick the model by planting a single fake sentence in the text. This fake sentence will use almost the exact same words, as the question, and structure as the real answer, but state the opposite fact. I believe this will noticeably drop the model’s accuracy, and in these cases, the model will confidently fall for the trap and report the fake answer as fact**, revealing that sometimes the extraction is driven by **surface-level lexical overlap**(i.e, two sentences share the exact same words on the surface, but they do not necessarily share the same meaning or logic) rather than by genuinely understanding the meaning of the passage.

Why This Direction

Retrieval quality metrics only tell us whether the pipeline found the right document,they do not make sure whether the reasoning component can be trusted once it has that document in hand. That is the failure mode that actually matters in production RAG systems, where retrieved context is never guaranteed to be clean. Instead of incrementally improving retrieval, we chose to stress-test the reasoning step directly, A qualitative failure, **where the model changes its answer even though the correct evidence remains completely untouched in the text**, is much more telling than a tiny, marginal shift in overall accuracy scores.

Method (brief)

For every question the baseline answered correctly, I cloned the sentence containing the gold answer, swapped only the answer span for a same surface level trap (a shifted year/number, or a decoy proper noun). I also introduced this fake fact with a contrasting cue (eg- ”However, other historical records dispute this and instead state that...”). The retrieved text stayed exactly the same across both test runs. Any change in performance is due entirely to the injected sentence, rather than a shift in the background context.

Condition	Exact Match (EM)	F1 Score
Baseline (clean context)	66.67%	77.67%
+ Adversarial distractor	60.67%	71.57%
$\triangle$ **(drop)**	**-6.00 pts**	**-6.10 pts**

Findings & Failure Analysis

Retrieval accuracy was unaffected (86.67% in both conditions), confirming the drop is due to the injected sentence rather than a retrieval failure. Out of all questions the baseline answered correctly, **2.67% were "confidently fooled"**, i.e, the model completely abandoned the correct answer and gave the fabricated one without any hesitation.

Another interesting finding is what the numbers don't show: the F1/EM drop ($\sim$6 points) is much larger than the fooled adoption rate (2.67%) would suggest. This proves that the main effect of the trick sentence is not simply brainwashing the model into believing the fake fact. Instead, a small number of clear traps are layered on top of a broader, subtler problem.

Imagine this: we chose a question the model initially answered correctly using the clean text. When we added a copy of that same sentence but swapped the real date or number for a fake one, the model switched its answer to the fake value, even though the original sentence was still right there in the text. Whats worse is this **failure was completely silent**. The model delivered the incorrect answer with the exact same confidence and smooth language as a correct one, showing no signs that it noticed any contradiction.

Next Steps (Testable)

1. **NLI-based contradiction filter**: Before generating any response, a small, efficient model (like roberta-large-mnli) scores every sentence in the text against the others. If any two sentences contradict each other with a confidence score above 80%, the system steps in. It can either delete the less relevant sentence, or show both conflicting facts to the user with a warning instead of quietly picking one.
2. **Contrastive fine tuning**: We will generate about 2,000 fake text examples using our existing script, where each example contains a clean context and an injected trap. We will then fine tune our base model using an efficient training method (LoRA). The key is using a specialized loss function that explicitly penalizes the model if it selects the fake answer over the real one. We will know this succeeds if the model's trap adoption rate on a separate test set drops below 1%, without losing more than 2 points of accuracy on clean text.